



Registre de Visites

Guia de Desplegament a Produccio

Instruccions per a l'equip de sistemes
Desplegament en Ubuntu Server amb Apache + Gunicorn + PostgreSQL

Versio	2.0
Data	Maig 2026
Entorn	Ubuntu Server 24.04 LTS + Apache 2.4 + Gunicorn + PostgreSQL
Stack	Python 3.11+ / FastAPI / SQLAlchemy async
Port backend	50003
DNS intern	visitesfc.agrienergia.local
Directori	/var/www/visites
Repositori	https://github.com/ohijazo/RegistreVisites
Destinatari	Equip de Sistemes - Farinera Coromina

1. Sobre l'aplicacio

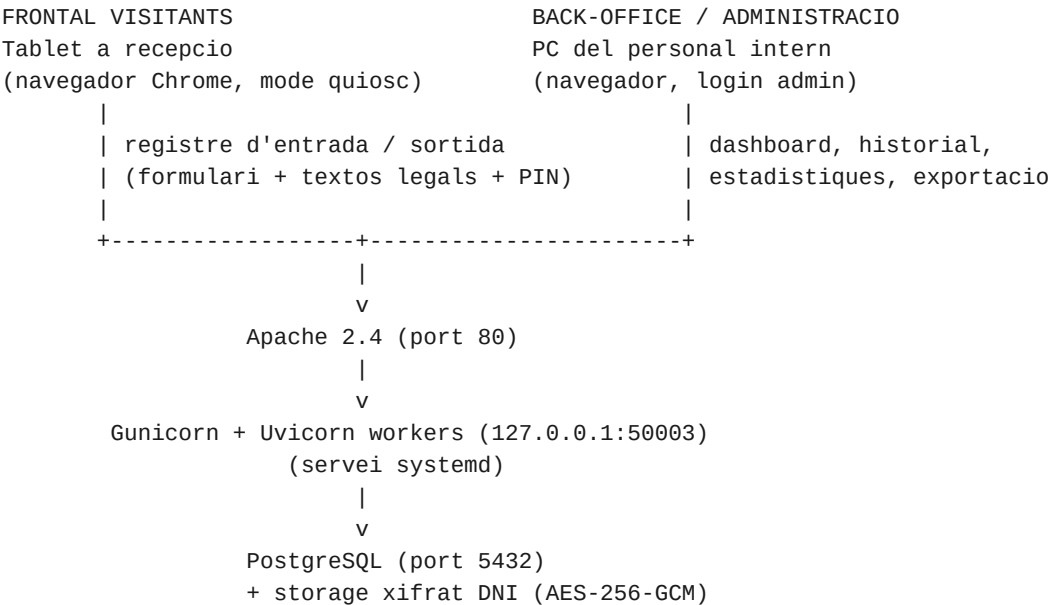
Registre de Visites es una aplicacio web en Python que digitalitza el registre d'entrada i sortida de visitants a les instal.lacions de Farinera Coromina. Substueix el registre en paper i compleix amb el RGPD (xifrat de DNI, consentiment registrat, purga automatica > 2 anys).

El sistema te dos canals d'accés:

- **Frontal de visitants** — Tablet fixa a recepcio en mode quiosc. El visitant es registra (idioma, dades, motiu, departament, acceptacio RGPD) i rep un PIN per a registrar la sortida quan se'n vagi.
- **Back-office / Administracio** — Accessible des de qualsevol PC del personal intern amb login. Permet veure visites actives en temps real, historial, estadistiques, gestionar departaments i textos legals, i exportar dades.

Opcionalment, l'app pot enviar una notificacio per email al departament que rep la visita.

Arquitectura



Components

Component	Versio	Funcio
Ubuntu Server	22.04 / 24.04 LTS	Sistema operatiu
Python	3.11+	Executa el backend (FastAPI async)
PostgreSQL	14+	Base de dades (asyncpg)
Apache	2.4+	Reverse proxy + headers de seguretat
Gunicorn	22+	Process manager (uvicorn workers)
Git	2.40+	Descarregar i actualitzar el codi

NOTA: A diferencia d'altres aplicacions internes, aquesta NO te frontend separat. Les pagines son generades pel servidor amb Jinja2 + HTMX + Tailwind (via CDN). No cal Node.js ni compilacio de cap fixer estatic.

2. Requisits de la Maquina Virtual

L'aplicacio es lleugera. Pot conviure perfectament amb altres aplicacions Python al mateix servidor (per exemple, la Preparacio de Comandes o les Fitxes Tecniques) compartint PostgreSQL i Apache.

Especificacions recomanades

Recurs	Minim	Recomanat
CPU	2 cores	2-4 cores
RAM	1 GB	2 GB (compartit amb altres apps)
Disc	10 GB	30 GB SSD
Sistema Operatiu	Ubuntu Server 22.04 LTS	Ubuntu Server 24.04 LTS
Xarxa	Connectivitat LAN	IP fixa a la xarxa local

Si ja hi ha un servidor Ubuntu amb PostgreSQL i Apache (com l'existent per a Preparacio de Comandes / Fitxes Tecniques), aquesta aplicacio s'instal.la pel mateix patro reaprofitant tota la infraestructura. No cal una maquina nova.

Consum estimat

Component	Consum	Observacions
Gunicorn + 2 workers	~150 MB RAM	Suficient per a ~20 visites simultanies
PostgreSQL (BD propia)	< 50 MB disc/any	Purga automatica > 2 anys (RGPD)
Codi de l'aplicacio	< 50 MB disc	Codi + plantilles + translations
Logs	< 100 MB/any	Rotacio via logrotate (Apache i app)

3. Instal·lacio de Paquets de Sistema

Si el servidor ja te aquestes versions instal·lades, es pot saltar aquest pas i passar directament al pas 4 (Base de Dades).

Pas 3.1 - Actualitzar el sistema

```
sudo apt update && sudo apt upgrade -y
```

Pas 3.2 - Instal·lar paquets necessaris

```
# Python (3.11+), pip, venv
sudo apt install -y python3 python3-pip python3-venv python3-dev

# PostgreSQL (si no esta instal·lat)
sudo apt install -y postgresql postgresql-contrib

# Apache (si no esta instal·lat) + moduls de proxy
sudo apt install -y apache2
sudo a2enmod proxy proxy_http headers rewrite expires

# Git i build tools (per a algunes dependencies natives)
sudo apt install -y git build-essential libpq-dev
```

Pas 3.3 - Verificar instal·lacions

```
python3 --version    # 3.11+
psql --version       # 14+
apache2 -v           # 2.4+
git --version        # 2.40+
```

4. Configurar PostgreSQL

L'aplicacio fa servir una base de dades propia anomenada **visites_db** dins del PostgreSQL existent. No interfereix amb cap altra base de dades del servidor.

Pas 4.1 - Crear la base de dades i l'usuari

```
sudo -u postgres psql <<EOF
CREATE DATABASE visites_db;
CREATE USER visites_user WITH PASSWORD 'CANVIAR_CONTRASENYA_SEGURA';
GRANT ALL PRIVILEGES ON DATABASE visites_db TO visites_user;

\c visites_db
ALTER DATABASE visites_db OWNER TO visites_user;
ALTER SCHEMA public OWNER TO visites_user;
GRANT ALL ON SCHEMA public TO visites_user;
GRANT CREATE ON SCHEMA public TO visites_user;
EOF
```

IMPORTANT (PostgreSQL 15+): l'esquema 'public' no es escriptible per defecte encara que es tinguin ALL PRIVILEGES sobre la base de dades. Cal canviar-ne el propietari (ALTER SCHEMA public OWNER) i donar CREATE. Si no, 'alembic upgrade head' fallara amb 'permission denied for schema public'.

IMPORTANT: Substituir 'CANVIAR_CONTRASENYA_SEGURA' per una contrasenya real. Evitar caracters que trenquen URLs (@, :, /, ?, #, ;, espais); si calen, codificar-los (@ -> %40, : -> %3A, etc.). Anotar la contrasenya per al .env.

Pas 4.2 - Verificar acces

```
# Provar el login amb el nou usuari
psql -U visites_user -d visites_db -h localhost -c 'SELECT version();'
# Hauria de mostrar la versio de PostgreSQL i sortir sense errors
```

5. Descarregar i Configurar l'Aplicacio

Pas 5.1 - Clonar el repositori

```
cd /var/www
sudo git clone https://github.com/ohijazo/RegistreVisites.git visites
sudo chown -R www-data:www-data /var/www/visites

# Carpeta de logs
sudo mkdir -p /var/log/visites
sudo chown www-data:www-data /var/log/visites
```

Pas 5.2 - Crear l'entorn virtual i instal·lar dependencies

```
cd /var/www/visites
sudo -u www-data python3 -m venv venv
sudo -u www-data venv/bin/pip install --upgrade pip
sudo -u www-data venv/bin/pip install -r requirements.txt
sudo -u www-data venv/bin/pip install gunicorn
```

Pas 5.3 - Generar claus criptografiques

Cal generar dues claus aleatòries: una per al xifrat AES-256-GCM dels DNI (**ENCRYPTION_KEY**) i una per a les sessions JWT i CSRF (**SECRET_KEY**).

```
# ENCRYPTION_KEY (32 bytes en base64)
python3 -c "import secrets,base64; print(base64.b64encode(secrets.token_bytes(32)).decode())"

# SECRET_KEY (cadena aleatòria llarga)
python3 -c "import secrets; print(secrets.token_urlsafe(48))"
```

CRITIC: Guardar ENCRYPTION_KEY en lloc segur (gestor de contrasenyes corporatiu). Si es perd aquesta clau, tots els DNI xifrats a la base de dades quedaran il·legibles per sempre. Mai canviar-la sense fer abans una migració dels registres.

Pas 5.4 - Crear el fitxer .env

```
cd /var/www/visites
sudo -u www-data cp .env.example .env
sudo nano .env
sudo chmod 600 .env
sudo chown www-data:www-data .env
```

Camps obligatoris (substituir els valors entre <>):

```
# Base de dades
DATABASE_URL=postgresql+asyncpg://visites_user:<CONTRASENYA_BD>@localhost/visites_db

# Claus generades al pas 5.3
ENCRYPTION_KEY=<CLAU_GENERADA_AES>
SECRET_KEY=<CLAU_GENERADA_SECRET>

# Empresa (apareix als textos legals i emails)
COMPANY_NAME=Farinera Coromina - Grup AE 1897
COMPANY_ADDRESS=<Adreça de l'empresa>
COMPANY_EMAIL=dpo@farineracoromina.com

# URL d'accés dels usuaris (s'usa per generar els QR)
```

```
BASE_URL=http://visitesfc.agrienergia.local
```

```
# Sessio admin i tablet  
SESSION_HOURS=8  
EXIT_TOKEN_HOURS=8  
KIOSK_RESET_SECONDS=30
```

```
# Entorn  
ENV=production  
DEBUG=false
```

Pas 5.5 - Executar migracions de base de dades

```
cd /var/www/visites  
sudo -u www-data venv/bin/alembic upgrade head  
# Hauria de crear totes les taules (visits, departments, legal_documents, etc.)
```

Pas 5.6 - Crear el primer usuari administrador

```
cd /var/www/visites  
sudo -u www-data venv/bin/python scripts/create_admin.py \  
    --email admin@farineracoromina.com \  
    --name "Administrador"
```

El script demanara la contrasenya per CLI. Aquesta sera la credencial inicial per accedir al panell **/admin/login**. Es pot canviar despres des del panell.

Pas 5.7 - Carregar el document legal RGPD inicial

```
cd /var/www/visites  
sudo -u www-data venv/bin/python scripts/seed_legal_doc.py
```

Aquest script carrega el text legal de visitants i subcontractistes (IT04.03 Bones Practiques) en els quatre idiomes (CA, ES, FR, EN). Despres es pot actualitzar des de **/admin/legal**.

6. Verificacio Manual de l'Aplicacio

Abans de configurar-la com a servei, conve provar que arrenca correctament des de la consola:

Pas 6.1 - Arrencar manualment

```
cd /var/www/visites  
sudo -u www-data venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 50003
```

Pas 6.2 - Provar des d'un altre terminal

```
# Health check  
curl http://localhost:50003/health  
# Hauria de retornar JSON amb status: ok  
  
# Pagina principal (redirigeix a /ca/)  
curl -I http://localhost:50003/  
# Hauria de retornar HTTP/1.1 302 o 307  
  
# Aturar amb Ctrl+C
```

Si els dos comandes retornen una resposta valida, l'aplicacio funciona. El seguent pas es deixar-la corrent de forma permanent com a servei systemd.

7. Configurar com a Servei (systemd)

Per garantir que l'aplicacio s'arrenqui automaticament amb la maquina i es reinicii si cau, es registra com a servei **systemd**. Es fa servir **Gunicorn** amb workers **Uvicorn** (el patro recomanat per FastAPI en produccio).

Pas 7.1 - Verificar permisos

```
# La carpeta de logs i el codi ja son de www-data del pas 5.1
# Verificar:
ls -ld /var/www/visites /var/log/visites
# Tots dos han de ser propietat de www-data:www-data
```

Pas 7.2 - Crear el fitxer de servei

```
sudo nano /etc/systemd/system/visites.service
```

Contingut:

```
[Unit]
Description=Registre de Visites - Backend
After=network.target postgresql.service
Requires=postgresql.service

[Service]
Type=exec
User=www-data
Group=www-data
WorkingDirectory=/var/www/visites
EnvironmentFile=/var/www/visites/.env
ExecStart=/var/www/visites/venv/bin/gunicorn app.main:app \
    --workers 2 \
    --worker-class uvicorn.workers.UvicornWorker \
    --bind 127.0.0.1:50003 \
    --timeout 30 \
    --access-logfile /var/log/visites/access.log \
    --error-logfile /var/log/visites/error.log
ExecReload=/bin/kill -s HUP $MAINPID
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Pas 7.3 - Activar i arrencar el servei

```
sudo systemctl daemon-reload
sudo systemctl enable visites
sudo systemctl start visites
sudo systemctl status visites
# Hauria de dir: active (running)
```

Comandes de gestio del servei

Comanda	Accio
<code>sudo systemctl start visites</code>	Arrencar el servei
<code>sudo systemctl stop visites</code>	Aturar el servei
<code>sudo systemctl restart visites</code>	Reiniciar el servei

Comanda	Accio
sudo systemctl status visites	Veure l'estat
sudo journalctl -u visites -f	Logs en temps real (journald)
sudo tail -f /var/log/visites/error.log	Logs d'errors de Unicorn
sudo systemctl disable visites	Desactivar arrencada automatica

8. Configurar Apache (Reverse Proxy)

Apache fa de reverse proxy davant del Gunicorn, gestiona els fitxers estàtics i afegeix els headers de seguretat. Segueix exactament el mateix patró que les aplicacions germanes **fitxes-tecniques** i **comandes-venda**.

Pas 8.1 - Activar moduls necessaris (si no s'han activat al pas 3.2)

```
sudo a2enmod proxy proxy_http headers rewrite expires
sudo systemctl restart apache2
```

Pas 8.2 - Crear el VirtualHost

```
sudo nano /etc/apache2/sites-available/visites.conf
```

Contingut:

```
<VirtualHost *:80>
    ServerName visitesfc.agrienergia.local

    # Headers de seguretat
    Header always set X-Frame-Options          "DENY"
    Header always set X-Content-Type-Options   "nosniff"
    Header always set Referrer-Policy          "strict-origin"

    # Limit de mida dels POST (formulari amb foto opcional)
    LimitRequestBody 2097152

    # Fitxers estàtics servits directament per Apache
    Alias /static/ /var/www/visites/static/
    <Directory /var/www/visites/static/>
        Require all granted
        Options -Indexes
        ExpiresActive On
        ExpiresDefault "access plus 30 days"
    </Directory>

    # Reverse proxy a Gunicorn (NO incloure /static/)
    ProxyPreserveHost On
    ProxyPass          /static/ !
    ProxyPass          / http://127.0.0.1:50003/
    ProxyPassReverse   / http://127.0.0.1:50003/

    # Capcaleres per a que el backend sapiga la IP real del visitant
    RequestHeader set X-Forwarded-Proto "http"

    ErrorLog  ${APACHE_LOG_DIR}/visites-error.log
    CustomLog ${APACHE_LOG_DIR}/visites-access.log combined
</VirtualHost>
```

Pas 8.3 - Activar el site i recarregar Apache

```
# Activar el site
sudo a2ensite visites.conf

# Validar la sintaxi
sudo apache2ctl configtest
# Hauria de dir: Syntax OK

# Recarregar Apache (sense interrompre les altres apps)
```

```
sudo systemctl reload apache2
```

Si el servidor ja te altres apps darrere d'Apache (com fitxes-tecniques o comandes-venda), aquest patro de sites-available + a2ensite permet afegir una app nova sense modificar res del que ja funciona.

9. Xarxa, DNS i Firewall

Firewall (UFW)

Si el firewall UFW esta actiu (comprovar amb **sudo ufw status**), nomes cal verificar que el port 80 i el SSH estan oberts. Cap port nou per a aquesta app.

```
sudo ufw allow ssh
sudo ufw allow 80/tcp
sudo ufw status
```

No cal obrir el port 50003 - Gunicorn nomes escolta a 127.0.0.1 i nomes Apache hi pot accedir.

DNS intern

Crear una entrada al servidor DNS de l'empresa per a que la tablet i els mobils dels visitants puguin accedir per nom (mes facil de codificar als QR que una IP):

```
Tipus:    A
Nom:      visitesfc.agrienergia.local
Valor:    <IP-DEL-SERVIDOR>      # p.ex. 192.168.1.50
```

La variable `BASE_URL` del fitxer `.env` ha de coincidir EXACTAMENT amb la URL que acaba a la tablet (p.ex. `http://visitesfc.agrienergia.local`). Si canvies aqui, actualitza tambe el `.env` i reinicia el servei: `sudo systemctl restart visites`.

10. Email de Notificacio (opcional)

Quan es crea una visita prevista des de l'admin, l'app pot enviar un avis per email al departament que rep la visita. Es opcional: si no es configura, l'aplicacio funciona igual pero sense enviar emails.

Hi ha dues opcions per integrar amb Microsoft 365 (recomanat respecte SMTP, que esta sent deprecad per Microsoft):

Opcio A - Power Automate (rapid)

Crear un flux instantani a **make.powerautomate.com** amb trigger HTTP que rebí un JSON i envii email amb el connector "Office 365 Outlook". L'app fa una crida POST a l'URL del webhook. Variables al .env:

```
EMAIL_BACKEND=power_automate
POWER_AUTOMATE_WEBHOOK_URL=https://prod-XX.westeurope.logic.azure.com:443/...
POWER_AUTOMATE_SECRET=<un-string-llarg-aleatori>
EXPECTED_NOTIFY_RECIPIENTS=cap@empresa.com, recepcio@empresa.com
```

Opcio B - Microsoft Graph API (recomanat a llarg termini)

Registrar una app a **entra.microsoft.com** amb permis d'aplicacio **Mail.Send**, restringida a una sola bustia amb ApplicationAccessPolicy. Variables al .env:

```
EMAIL_BACKEND=graph_ms
MS_TENANT_ID=<directory_tenant_id>
MS_CLIENT_ID=<application_client_id>
MS_CLIENT_SECRET=<valor_del_secret>
MS_SENDER_EMAIL=coromina@empresa.com
EXPECTED_NOTIFY_RECIPIENTS=cap@empresa.com, recepcio@empresa.com
```

Veure el detall pas a pas (apartats 11 i 12) al fitxer DEPLOY.md del repositori. Despres de canviar el .env, sempre reiniciar: `sudo systemctl restart visites`.

Verificacio

Crear una visita prevista des de **/admin/expected/new** i comprovar:

- L'email arriba al destinatari (mirar tambe la carpeta Spam)
- **/admin/audit-logs?action=expected_visit_email_sent_auto** mostra la traca
- Si falla, mirar **action=expected_visit_email_failed_auto** per veure l'error

11. Tasques Programades (cron)

Dues tasques nocturnes son indispensables: tancar visites que s'han quedat obertes (visitant ha marxat sense fer checkout) i purgar registres de mes de 2 anys segons RGPD (article 5.1.e).

Pas 11.1 - Editar el crontab de www-data

```
sudo crontab -u www-data -e
```

Afegir aquestes línies (les tres primeres son variables per escurcar la resta - es el patró estàndard de crontab):

```
PY=/var/www/visites/venv/bin/python
APP=/var/www/visites
LOGS=/var/log/visites

# Auto-checkout nocturn: tanca visites obertes > 12h
55 23 * * * $PY $APP/scripts/auto_close_visits.py >> $LOGS/auto_close.log 2>&1

# Neteja RGPD: elimina visites > 2 anys (article 5.1.e)
0 3 * * * $PY $APP/scripts/purge_old_visits.py >> $LOGS/purge.log 2>&1
```

Backup diari de la base de dades

Recomanat per política de l'empresa. Backup automàtic cada nit i retenció de 30 dies:

```
sudo mkdir -p /opt/backups/visites
sudo chown postgres:postgres /opt/backups/visites
```

Crear el fitxer de cron de sistema:

```
sudo nano /etc/cron.d/visites-backup
```

Contingut (atenció: els \% son necessaris perquè cron interpreta % com a salts de línia):

```
BACKUP_DIR=/opt/backups/visites

# Backup diari a les 02:00
0 2 * * * postgres pg_dump visites_db | \
    gzip > $BACKUP_DIR/visites_$(date +%Y%m%d).sql.gz

# Eliminar backups > 30 dies (a les 03:00)
0 3 * * * root find $BACKUP_DIR -name "*.sql.gz" -mtime +30 -delete
```

12. Verificacio Final

Checklist de verificacio

#	Verificacio	Com comprovar-ho
1	PostgreSQL actiu	sudo systemctl status postgresql
2	Aplicacio activa	sudo systemctl status visites
3	Apache actiu	sudo systemctl status apache2
4	Health check	curl http://localhost:50003/health
5	Acces des de tablet	Obrir http://visitesfc.agrienergia.local al navegador
6	Login admin	http://visitesfc.agrienergia.local/admin/login
7	Formulari visitant	Completar registre des de la tablet
8	QR de sortida funciona	Escanejar el QR amb el mobil
9	PIN de sortida funciona	http://visitesfc.agrienergia.local/checkout amb el PIN
10	Servei resisteix reboot	sudo reboot i verificar acces despres

Prova completa des d'un dispositiu

Des d'una tablet o PC de la xarxa local, obrir el navegador i anar a:

`http://visitesfc.agrienergia.local`

Hauria d'apareixer la pantalla de seleccio d'idioma (CA / ES / FR / EN). Completar el flux de registre i verificar que es genera un QR i un PIN. Despres, des de **/checkout** introduir el PIN per registrar la sortida.

13. Manteniment i Actualitzacions

Actualitzar l'aplicació

Quan es lliuri una nova versió:

```
cd /var/www/visites

# 1. Descarregar canvis
sudo -u www-data git pull

# 2. Actualitzar dependències (si requirements.txt ha canviat)
sudo -u www-data /var/www/visites/venv/bin/pip install -r requirements.txt

# 3. Aplicar noves migracions (si n'hi ha)
sudo -u www-data /var/www/visites/venv/bin/alembic upgrade head

# 4. Restaurar permisos i reiniciar
sudo chown -R www-data:www-data /var/www/visites
sudo systemctl restart visites

# 5. Verificar
sudo systemctl status visites
curl http://localhost:50003/health
```

Logs - ubicacions

Log	Ubicació
Gunicorn - acces	/var/log/visites/access.log
Gunicorn - errors	/var/log/visites/error.log
systemd (journald)	sudo journalctl -u visites -f
Apache - acces	/var/log/apache2/visites-access.log
Apache - errors	/var/log/apache2/visites-error.log
Cron - auto-checkout	/var/log/visites/auto_close.log
Cron - purga RGPD	/var/log/visites/purge.log
PostgreSQL	/var/log/postgresql/

Resolució de problemes

Sintoma	Causa probable / Solució
Servei no arrenca	Mirar /var/log/visites/error.log o journalctl -u visites -n 50
Error 502/503 a Apache	Gunicorn caigut: sudo systemctl restart visites
Error de BD	Verificar PostgreSQL: sudo systemctl status postgresql
Pàgina no carrega	Validar Apache: sudo apache2ctl configtest, comprovar firewall
Error de xifrat de DNI	Verificar que ENCRYPTION_KEY al .env coincideix amb la que es va usar inicialment
DNI sembla il·legible	Mai canviar ENCRYPTION_KEY després del primer registre. Restaurar backup
QR no funciona des del mòbil	Verificar que BASE_URL al .env és accessible des de la xarxa Wi-Fi del visitant
Tablet es queda penjada	KIOSK_RESET_SECONDS al .env controla el reset automàtic

14. Resum i Consideracions de Seguretat

Resum de passos

Pas	Accio	Temps estimat
1	Preparar maquina virtual Ubuntu Server 24.04	10 min
2	Instal·lar paquets (apt install)	10 min
3	Crear BD i usuari PostgreSQL	5 min
4	Clonar repositori i configurar .env	10 min
5	Migracions + admin + document legal	5 min
6	Verificacio manual amb uvicorn	5 min
7	Crear servei systemd (Gunicorn + Uvicorn)	5 min
8	Configurar Apache (reverse proxy)	5 min
9	DNS intern + firewall	5 min
10	Email (opcional, Power Automate o Graph API)	15-30 min
11	Cron de manteniment + backup diari	5 min
12	Verificacio final amb tablet	10 min

Temps total estimat: ~90 minuts (sense comptar Email amb Graph)

Consideracions de seguretat i RGPD

Aspecte	Implementacio
Xifrat de DNI	AES-256-GCM amb IV unic per registre. Clau a ENCRYPTION_KEY (.env)
Consentiment registrat	legal_document_id + accepted_at + IP + user-agent a cada visita
Dret de supressio	Boto a /admin/visits/{id} per eliminar registre individualment
Purga automatica	Script purge_old_visits.py via cron, elimina > 2 anys
Auditoria d'accés a DNI	Cada desxifrat queda registrat a audit_logs (admin_id, IP, timestamp)
Sessions admin	JWT en cookie HttpOnly + SameSite=Strict. Expiracio: SESSION_HOURS
CSRF	Token double-submit a tots els POST/PUT/DELETE
Rate limiting	Max 10 POST/minut/IP al formulari de registre (slowapi)
Headers de seguretat	X-Frame-Options, X-Content-Type-Options, CSP, Referrer-Policy (a l'app i Apache mod_headers)
DNI als logs	Apache (combined log) NO registra el cos del POST. App tampoc loga el camp id_document

Contacte

Per a qualsevol incidència tècnica relacionada amb el desplegament, contactar amb el responsable del projecte:

Repositori	https://github.com/ohijazo/RegistreVisites
Documentacio	/var/www/visites/DEPLOY.md (versio detallada al repositori)
Empresa	Farinera Coromina - Grup AE 1897

